

Mini-Projects: Introduction to Delphi tools

These interactive activities naturally build on each other, but can also be done in isolation if you only have a few minutes to stop by.

Let the host know if you're interested in a particular section or topic! They can set you up with all precursor code.

Follow-along with the ★ items, or choose your own path!

① symbols offer questions to help you get to know the data or system better.

✂ symbols provide “Side Quests” for exploring additional topics or diving deeper into a particular function



Access this worksheet later!

MINI-PROJECT 3: Customizing models

The R package `epipredict` provides quick-start forecasters meant to run with little modification, as well as a framework for customizing and developing your own models. `epipredict` documentation and vignettes are available at cmu-delphi.github.io/epipredict.

3.0 Load the required packages

The `{InsightNetApr26}` package ensures all required Delphi tools and their correct versions/branches are installed for the purpose of this session. You can run the code below to install it.

```
if (!requireNamespace("pak", quietly = TRUE)) install.packages("pak")
if (!requireNamespace("InsightNetApr26", quietly = TRUE)) {

  pak::pkg_install("cmu-delphi/InsightNet-apr-2026/InsightNetApr26")
}
InsightNetApr26::verify_setup()

library(epidatr)
```

```
library(epiprocess)
library(epipredict)
library(dplyr)
library(ggplot2)
library(parsnip)
```

3.1 Basic Modelling

★ We'll use the built-in `covid_case_death_rates` dataset provided by the `epidatasets` package, which automatically loads with the other Delphi tools. [Hint: You can get a partial view of the dataset by entering `covid_case_death_rates` in the R console.]

- Filter out the observations after August 1, 2021 to create a training set.

① In a real-world scenario, reporting delays mean that recent data is often missing or subject to revision. If you accidentally trained your model on data that wasn't actually available at the time of the forecast, how would this "data leakage" give you a misleading impression of your model's accuracy?

- Filter to four states of your choosing and use `autoplot()` to visualize the `death_rate` and `case_rate` columns.
- Let's fit an autoregressive model on our training set using lagged values of the `death_rate` column as features with `arx_forecaster()`. [Hint: you need to provide the dataset and the outcome argument to `arx_forecaster()`]

✂ Why is an autoregressive model a good choice? Learn more about them here:
<https://www.geeksforgeeks.org/data-analysis/autoregressive-ar-model-for-time-series-forecasting/>

- When you run a forecaster, it returns an object containing several useful components.
 - Use `autoplot(your_forecast)` for a quick plot
 - Use `your_forecast$predictions` on the object returned by the forecast to view the results. Use `pivot_quantiles_wider()` to expand the quantile distribution column into individual columns.

- Use `your_forecast$epi_workflow` to access the underlying model. To see exactly how much weight the model gave to each lag, pass this object to `hardhat::extract_fit_engine()`.

① How heavily does the model weigh data from exactly one week ago compared to yesterday? Try running `hardhat::extract_fit_engine(your_forecast$epi_workflow)` to examine the fitted coefficients for each lag. Which lag contributes the most to the forecast?

✂ Compare your autoregressive model fitted with `arx_forecaster()` to a simple model. The flatline forecaster is a common baseline that assumes the future will be exactly the last observed value, with uncertainty that increases as the forecast horizon grows. Fit a flatline model using `flatline_forecaster()` and examine the predictions to see how the quantiles behave.

3.2 Model Customization

★ While “canned” forecasters such as `arx_forecaster()` are useful for establishing baselines, you will often want to customize your models. Use the `arx_forecaster()` arguments to customize your model.

- So far, we have only used the outcome itself as a predictor. However, many signals serve as leading indicators. For example, changes in case rates often precede changes in death rates. Use both `case_rate` and `death_rate` as predictors of `death_rate` in `arx_forecaster()`. [Hint: use the `predictors` argument]
- By default, `arx_forecaster()` uses linear regression. Swap this for `quantile_reg()`.
- Further customize your model by modifying the `arx_args_list()` arguments via the `args_list` parameter in `arx_forecaster()`. [Hint: you can access the `arx_args_list()` documentation by typing `?arx_args_list` in R.]
 - Use the `lags` argument in `arx_args_list()` to specify a vector of integers representing the historical days for each indicator.

Hint: `lags = c(0, 7, 14)` tells the model to use data from today, one week ago, and two weeks ago as predictors. If you are using multiple indicators as predictors, you can provide a list of vectors (e.g., `lags = list(c(0, 7), c(0, 14))`) where each vector corresponds to one of your indicators.

- Set the `ahead` argument to 28.

✂ Create a forecast trajectory over a range of dates and plot it with `autoplot()`. This creates a full trajectory showing how uncertainty grows with the horizon. [Hint: see this example on creating models for multiple ahead arguments:

cmu-delphi.github.io/epipredict/articles/epipredict.html#generating-multiple-aheads]

3.3 Use Delphi data in other model types

★ We will use `arx_classifier()` to predict disease growth categories. This model doesn't just predict a value, but it predicts the probability that the growth rate falls into specific categories.

- Create a classifier to predict whether `death_rate` is ≤ 0.01 or > 0.01 using `case_rate` and `death_rate` as predictors.

Hint: to configure the classifier, we use `arx_class_args_list()` as input in the `args_list` argument. The key parameter is `breaks`, which defines the thresholds for our categories.

Interested in trying this sort of analysis on your own data and building your own custom epiworkflows? For more information on this, visit:

<https://cmu-delphi.github.io/epipredict/index.html>

Example solutions!

